CS3014 - Estructuras de Datos Avanzadas

Dynamic Optimality III
Semana 6

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

16 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Representar ejecuciones de BST como conjuntos de puntos

Objetivos de la sesión



- Representar ejecuciones de BST como conjuntos de puntos
- Satisfacción arboral, y su equivalencia con BSTs válidos

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Representar ejecuciones de BST como conjuntos de puntos
- Satisfacción arboral, y su equivalencia con BSTs válidos
- El algoritmo Greedy, y su conjetura central

Objetivos de la sesión



- Representar ejecuciones de BST como conjuntos de puntos
- Satisfacción arboral, y su equivalencia con BSTs válidos
- El algoritmo Greedy, y su conjetura central
- Cómo simularlo *online*

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dónde quedamos

Splay Trees son un candidato **concreto** a optimalidad dinámica — pero seguimos sin saber si lo son, ni tenemos forma de comparar candidatos entre sí de forma rigurosa.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dónde quedamos

Splay Trees son un candidato **concreto** a optimalidad dinámica — pero seguimos sin saber si lo son, ni tenemos forma de comparar candidatos entre sí de forma rigurosa.

¡Piénsalo!

Todo lo que hace un algoritmo de BST, en el fondo, es **tocar** ciertos nodos en cierto orden. ¿Se podría representar **eso** de una forma más simple, despojada de la estructura de árbol?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dónde quedamos

Splay Trees son un candidato **concreto** a optimalidad dinámica — pero seguimos sin saber si lo son, ni tenemos forma de comparar candidatos entre sí de forma rigurosa.

¡Piénsalo!

Todo lo que hace un algoritmo de BST, en el fondo, es **tocar** ciertos nodos en cierto orden. ¿Se podría representar **eso** de una forma más simple, despojada de la estructura de árbol?

La idea de hoy

Sí: como un conjunto de **puntos** en el plano (clave, tiempo).

Representación geométrica

De accesos a puntos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

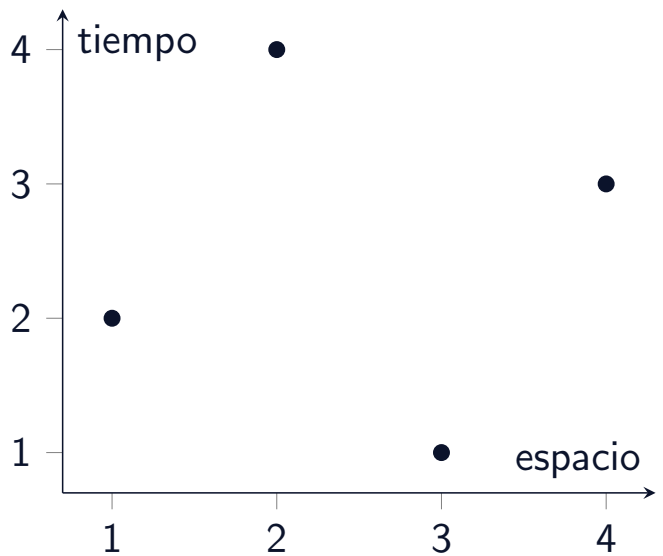
La idea

Dada una secuencia de accesos $x_1, \ldots, x_m$ sobre claves $\{1, \ldots, n\}$, dibuja un punto (x_i, i) por cada acceso — eje horizontal: la clave; eje vertical: el tiempo.

De accesos a puntos

La idea

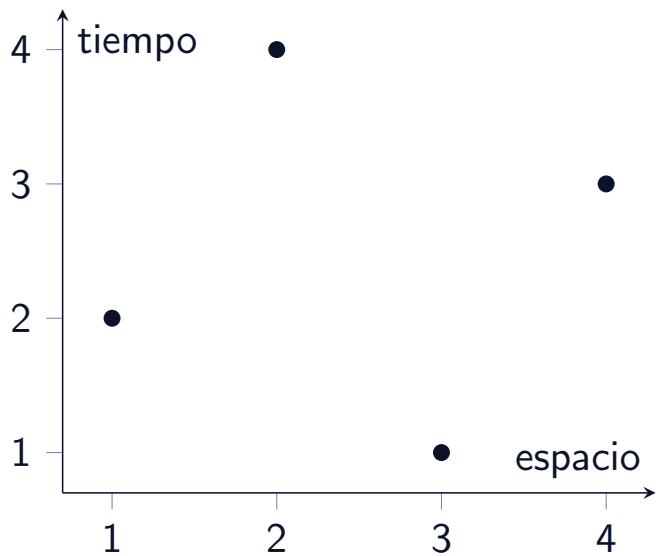
Dada una secuencia de accesos $x_1, \ldots, x_m$ sobre claves $\{1, \ldots, n\}$, dibuja un punto (x_i, i) por cada acceso — eje horizontal: la clave; eje vertical: el tiempo.



De accesos a puntos

La idea

Dada una secuencia de accesos $x_1, \ldots, x_m$ sobre claves $\{1, \ldots, n\}$, dibuja un punto (x_i, i) por cada acceso — eje horizontal: la clave; eje vertical: el tiempo.



Ejemplo: claves $\{1, 2, 3, 4\}$, secuencia de accesos $3, 1, 4, 2$.

¿Qué representa cada punto?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¡Piénsalo!

Al ejecutar realmente un BST para esta secuencia, cada acceso **toca** varios nodos (todo el camino recorrido desde la raíz), no solo el nodo buscado. ¿Cómo se ve eso en el dibujo?

¿Qué representa cada punto?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

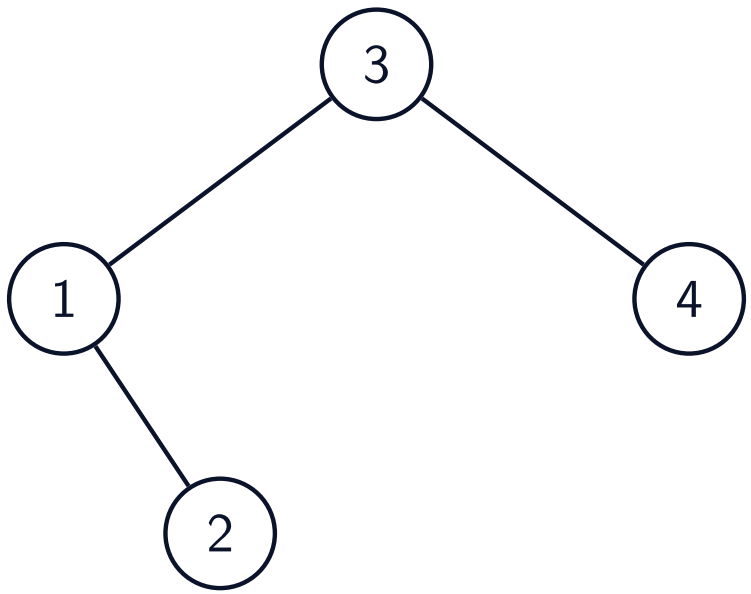
¡Piénsalo!

Al ejecutar realmente un BST para esta secuencia, cada acceso **toca** varios nodos (todo el camino recorrido desde la raíz), no solo el nodo buscado. ¿Cómo se ve eso en el dibujo?

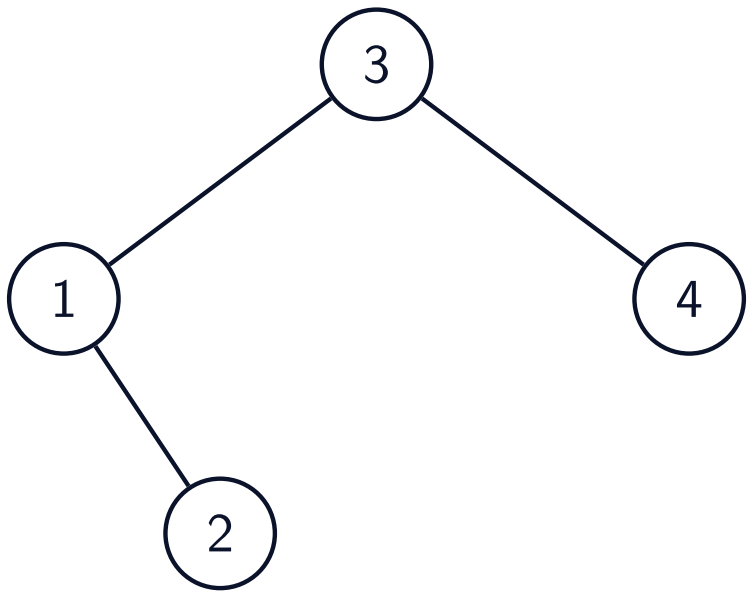
Un punto por cada nodo tocado

Si el acceso en el tiempo i toca el nodo x, dibujamos el punto (x, i) — **no** solo un punto por acceso, sino uno por cada nodo visitado en el camino de esa búsqueda.

Ejemplo completo

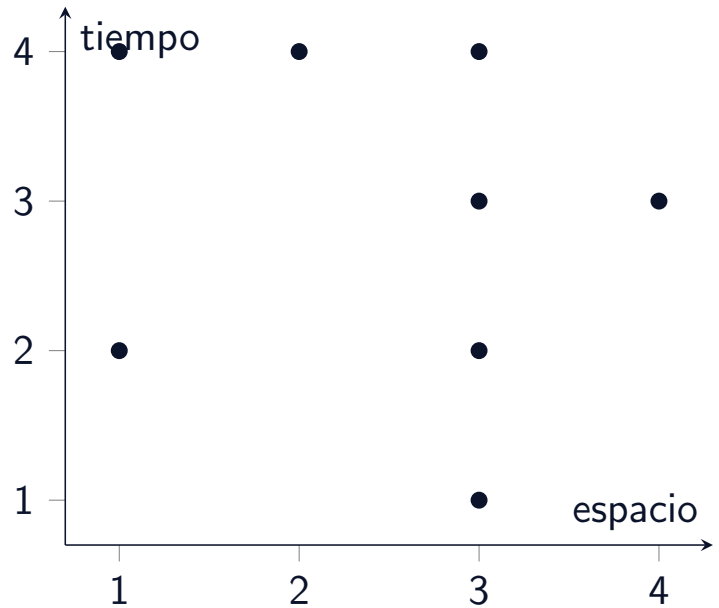


Ejemplo completo

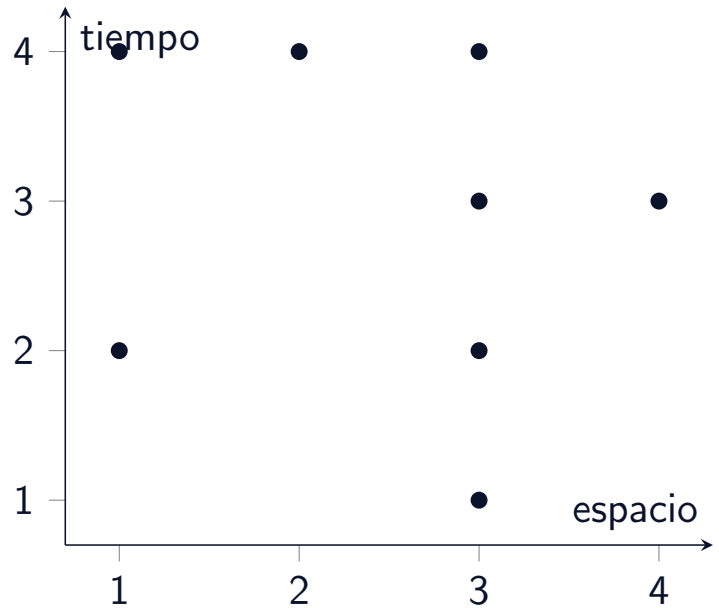


Un árbol estático posible para $\{1, 2, 3, 4\}$. Accediendo $3, 1, 4, 2$ en orden: acceso a 3 toca $\{3\}$; acceso a 1 toca $\{3, 1\}$; acceso a 4 toca $\{3, 4\}$; acceso a 2 toca $\{3, 1, 2\}$.

El conjunto de puntos resultante



El conjunto de puntos resultante



El **costo** del algoritmo = el número total de puntos — nunca hace falta tocar un nodo más de $O(1)$ veces por búsqueda, así que esto está dentro de un factor constante del costo real.

Satisfacción arboral

La condición clave

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un conjunto de puntos es **arboralmente satisfecho** si: para todo par de puntos que **no** comparten línea horizontal ni vertical, existe un **tercer** punto dentro del rectángulo que definen (o en su borde).

La condición clave

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un conjunto de puntos es **arboralmente satisfecho** si: para todo par de puntos que **no** comparten línea horizontal ni vertical, existe un **tercer** punto dentro del rectángulo que definen (o en su borde).

Puedes verificar que el conjunto de la diapositiva anterior cumple esto para todo par de puntos.

El teorema central

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Un conjunto de puntos (x_i, i) es arboralmente satisfecho **si y solo si** corresponde a una ejecución **válida** de algún algoritmo de BST para la secuencia $x_1, \ldots, x_m$.

El teorema central

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Un conjunto de puntos (x_i, i) es arboralmente satisfecho **si y solo si** corresponde a una ejecución **válida** de algún algoritmo de BST para la secuencia $x_1, \ldots, x_m$.

Corolario

La ejecución **óptima** de BST es equivalente al conjunto arboralmente satisfecho **más chico** que contiene a los puntos de entrada.

El teorema central

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Un conjunto de puntos (x_i, i) es arboralmente satisfecho **si y solo si** corresponde a una ejecución **válida** de algún algoritmo de BST para la secuencia $x_1, \ldots, x_m$.

Corolario

La ejecución **óptima** de BST es equivalente al conjunto arboralmente satisfecho **más chico** que contiene a los puntos de entrada.

Convertimos un problema sobre **árboles** en un problema puramente **geométrico**: ¿cuál es el menor superconjunto arboralmente satisfecho de un conjunto de puntos dado?

Prueba: BST válido $\Rightarrow$ arboralmente satisfecho



Dos puntos (x, i), (y, j), con $x < y$, $i < j$

Sea $\text{lca}_t(a, b)$ el ancestro común más bajo de a, b justo antes del tiempo t. Tres casos:

Prueba: BST válido $\Rightarrow$ arboralmente satisfecho

Dos puntos (x, i), (y, j), con $x < y$, $i < j$

Sea $\text{lca}_t(a, b)$ el ancestro común más bajo de a, b justo antes del tiempo t. Tres casos:

Casos

Prueba: BST válido $\Rightarrow$ arboralmente satisfecho

Dos puntos (x, i), (y, j), con $x < y$, $i < j$

Sea $\text{lca}_t(a, b)$ el ancestro común más bajo de a, b justo antes del tiempo t. Tres casos:

Casos

- Si $\text{lca}_i(x, y) \neq x$: usa el punto $(\text{lca}_i(x, y), i)$ — se tocó junto con x.

Prueba: BST válido $\Rightarrow$ arboralmente satisfecho

Dos puntos (x, i), (y, j), con $x < y$, $i < j$

Sea $\text{lca}_t(a, b)$ el ancestro común más bajo de a, b justo antes del tiempo t. Tres casos:

Casos

- Si $\text{lca}_i(x, y) \neq x$: usa el punto $(\text{lca}_i(x, y), i)$ — se tocó junto con x.
- Si $\text{lca}_j(x, y) \neq y$: usa $(\text{lca}_j(x, y), j)$, análogamente.

Prueba: BST válido $\Rightarrow$ arboralmente satisfecho

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos puntos (x, i), (y, j), con $x < y$, $i < j$

Sea $\text{lca}_t(a, b)$ el ancestro común más bajo de a, b justo antes del tiempo t. Tres casos:

Casos

- Si $\text{lca}_i(x, y) \neq x$: usa el punto $(\text{lca}_i(x, y), i)$ — se tocó junto con x.
- Si $\text{lca}_j(x, y) \neq y$: usa $(\text{lca}_j(x, y), j)$, análogamente.
- Si ninguno: x es ancestro de y antes de i, y y es ancestro de x antes de j — en algún momento k intermedio, y tuvo que rotar por encima de x; usa (y, k).

Prueba: arboralmente satisfecho $\Rightarrow$ BST válido

La otra dirección es constructiva: se organiza el conjunto de puntos como un *treap (cartesian tree)* ordenado (en cada momento) por el **próximo tiempo de toque** de cada nodo. La satisfacción arboral garantiza que, en cada instante, los nodos ya tocados forman un subárbol **conexo** en la parte superior — exactamente la forma que necesita un BST real para ejecutar esa secuencia de accesos.

Prueba: arboralmente satisfecho $\Rightarrow$ BST válido

La otra dirección es constructiva: se organiza el conjunto de puntos como un *treap (cartesian tree)* ordenado (en cada momento) por el **próximo tiempo de toque** de cada nodo. La satisfacción arboral garantiza que, en cada instante, los nodos ya tocados forman un subárbol **conexo** en la parte superior — exactamente la forma que necesita un BST real para ejecutar esa secuencia de accesos.

Pregunta abierta

¿Cuál es la complejidad computacional de encontrar el menor conjunto arboralmente satisfecho? ¿Existe una $O(1)$-aproximación?

El algoritmo Greedy

Construyendo, fila por fila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Greedy

Procesa el conjunto de puntos **una fila (tiempo) a la vez**. En cada fila, agrega los puntos **mínimos** necesarios para que el conjunto acumulado hasta ahí quede arboralmente satisfecho.

Construyendo, fila por fila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Greedy

Procesa el conjunto de puntos **una fila (tiempo) a la vez**. En cada fila, agrega los puntos **mínimos** necesarios para que el conjunto acumulado hasta ahí quede arboralmente satisfecho.

¡Piénsalo!

Esto construye la ejecución de BST **fila por fila** — ¿en qué se parece esto a ejecutar un BST real, accediendo una consulta a la vez?

Construyendo, fila por fila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Greedy

Procesa el conjunto de puntos **una fila (tiempo) a la vez**. En cada fila, agrega los puntos **mínimos** necesarios para que el conjunto acumulado hasta ahí quede arboralmente satisfecho.

¡Piénsalo!

Esto construye la ejecución de BST **fila por fila** — ¿en qué se parece esto a ejecutar un BST real, accediendo una consulta a la vez?

Respuesta

Es exactamente análogo: en cada momento, agregar "lo mínimo necesario" para mantenerse arboralmente satisfecho imita rotar/reorganizar lo mínimo necesario para completar la búsqueda actual.

La conjetura central

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Conjetura (abierta)

$\textsc{Greedy} = O(OPT)$ — o incluso $\textsc{Greedy} = OPT + O(m)$.

La conjetura central

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Conjetura (abierta)

$\textsc{Greedy} = O(OPT)$ — o incluso $\textsc{Greedy} = OPT + O(m)$.

Si esto fuera cierto, tendríamos un algoritmo $O(1)$-**competitivo** — es decir, **optimalidad dinámica resuelta**. Sigue sin demostrarse ni refutarse.

Simulación *online*

¿Y si no conocemos el futuro?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Un algoritmo *online* para mantener conjuntos arboralmente satisfechos implica un algoritmo de BST *online* con **desaceleración** $O(1)$.

¿Y si no conocemos el futuro?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

Un algoritmo *online* para mantener conjuntos arboralmente satisfechos implica un algoritmo de BST *online* con **desaceleración** $O(1)$.

Corolario

Si Greedy es $O(OPT)$, entonces tenemos un algoritmo de BST dinámicamente óptimo — **y** *online*.

La idea de la simulación

Los nodos tocados en un acceso se guardan en un ***split tree***: una estructura que puede mover un nodo a la raíz y luego **separarlo** en dos árboles, todo en $O(1)$ amortizado. Organizando el BST real como un *treap* de estos *split trees* (ordenado por el nodo tocado más recientemente), cada *split tree* puede acceder en $O(1)$ a su predecesor y sucesor en el árbol completo — suficiente para simular las decisiones de GREEDY con solo un factor constante de más trabajo.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Una ejecución de BST se representa como un conjunto de puntos (x_i, i): uno por cada nodo tocado en cada acceso.
- Un conjunto de puntos es **arboralmente satisfecho** si todo rectángulo no degenerado que definen dos puntos contiene un tercero — y esto es **equivalente** a corresponder a una ejecución BST válida.
- El BST óptimo = el menor conjunto arboralmente satisfecho que contiene la entrada: un problema puramente geométrico.
- GREEDY construye ese conjunto fila por fila; se conjetura (sin demostrar) que es $O(OPT)$.
- Vía *split trees*, GREEDY se puede simular *online* con solo $O(1)$ de desaceleración.

Logros de la sesión



Logros de la sesión

- Sabemos representar cualquier ejecución de BST como un conjunto de puntos.

Logros de la sesión

- Sabemos representar cualquier ejecución de BST como un conjunto de puntos.
- Sabemos verificar (y probar) la equivalencia entre satisfacción arboral y ejecuciones BST válidas.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos representar cualquier ejecución de BST como un conjunto de puntos.
- Sabemos verificar (y probar) la equivalencia entre satisfacción arboral y ejecuciones BST válidas.
- Sabemos ejecutar GREEDY, y qué dice su conjetura central.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos representar cualquier ejecución de BST como un conjunto de puntos.
- Sabemos verificar (y probar) la equivalencia entre satisfacción arboral y ejecuciones BST válidas.
- Sabemos ejecutar GREEDY, y qué dice su conjetura central.
- Entendemos, a alto nivel, cómo simular GREEDY *online* con *split trees*.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Demaine, E. D., Harmon, D., Iacono, J., Kane, D. M., & Pătraşcu, M. (2009). The geometry of binary search trees. *Proceedings of SODA 2009*, 496–505.
- Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. `https://courses.csail.mit.edu/6.851/spring21/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.